

FTML: Project Report

Adrien Capitaine

Brieuc Crosson

Nathan Champagne

June 2026

Contents

1	Bayes estimator and Bayes risk	2
2	Bayes risk with the absolute loss	5
3	Expected value of the empirical risk for OLS	9
4	Regression on a given dataset	13
5	Classification on a given dataset	14
6	Application of supervised learning: credit-card fraud detection	16
7	Application of unsupervised learning: Spotify “vibe” clustering	18

1 Bayes estimator and Bayes risk

Code: `exercise_1_bayes/bayes_simulation.py`.

Question 1 (M)

We choose the following supervised learning setting: counting radioactive decays. A Geiger counter is placed next to a radioactive source and counts the decays during a short time window.

- **Input** $\mathcal{X} = [0.5, 2.0] \subset \mathbb{R}$: the observation duration x , in seconds (s).
- **Output** $\mathcal{Y} = \mathbb{R}_+$: the prediction $f(x)$, a real expected number of decays. The label itself is an integer count, $Y \in \mathbb{N}$.
- **Loss**: squared loss $\ell(y, f(x)) = (y - f(x))^2$ (unit: counts²).

Note. We use $\mathbb{R}_+$ rather than $\mathbb{N}$ as the prediction space. The observed labels Y are integers, but the best prediction under the squared loss is the conditional mean $\mathbb{E}[Y \mid X = x]$, which is a real number.

Laws of X and Y :

- **Marginal law of X (uniform).** We take X uniform on $[0.5, 2.0]$,

$$X \sim \mathcal{U}([0.5, 2.0]), \quad f_X(x) = \frac{1}{2.0 - 0.5} = \frac{2}{3}$$

We choose the uniform law because the duration may be set anywhere in the range with no preferred value.

- **Conditional law of Y given X (Poisson).** Given $X = x$, the count Y follows a Poisson law of mean $\lambda(x) = \Phi x$, with activity $\Phi = 50$ decays/s:

$$Y \mid X = x \sim \text{Poisson}(\lambda(x)), \quad P(Y = y \mid X = x) = \frac{\lambda(x)^y e^{-\lambda(x)}}{y!} = \frac{(50x)^y e^{-50x}}{y!}, \quad y \in \mathbb{N}$$

We choose the Poisson law because counting rare, independent decays over a fixed window follows exactly a Poisson law. Its mean is the activity Φ multiplied by the duration x , hence $\lambda(x) = \Phi x$.

Joint distribution: We obtain the joint distribution of (X, Y) by multiplying the two laws above, using the factorization $P(X, Y) = P(X) P(Y \mid X)$:

$$f_{X,Y}(x, y) = f_X(x) P(Y = y \mid X = x) = \frac{2}{3} \cdot \frac{(50x)^y e^{-50x}}{y!}, \quad x \in [0.5, 2.0], \quad y \in \mathbb{N}$$

Bayes predictor: For the squared loss, the course gives the Bayes predictor as the conditional mean (eq. (41), p. 20):

$$f^*(x) = \mathbb{E}[Y \mid X = x]$$

The mean of a Poisson law is equal to its parameter, $\mathbb{E}[Y \mid X = x] = \lambda(x) = \Phi x$ with $\Phi = 50$, which gives

$$f^*(x) = \mathbb{E}[Y \mid X = x] = \lambda(x) = \Phi x = 50x$$

Bayes risk: We start from the course definition of the Bayes risk (eq. (40), p. 19), for a general loss ℓ :

$$R^* = \mathbb{E}_X \left[\inf_{y \in \mathbb{R}_+} \mathbb{E}_Y [\ell(Y, y) | X] \right]$$

Specialising to the squared loss $\ell(y, f(x)) = (y - f(x))^2$ gives

$$R^* = \mathbb{E}_X \left[\inf_{y \in \mathbb{R}_+} \mathbb{E}_Y [(Y - y)^2 | X] \right]$$

For each x , the minimum over y is reached at the conditional mean $y = \mathbb{E}[Y | X = x]$ (eq. (41), p. 20). If we put this value back, we get

$$\inf_{y \in \mathbb{R}_+} \mathbb{E}_Y [(Y - y)^2 | X = x] = \mathbb{E}_Y [(Y - \mathbb{E}[Y | X = x])^2 | X = x] = \text{Var}(Y | X = x)$$

which is the definition of the conditional variance. Taking the expectation over X ,

$$R^* = \mathbb{E}_X [\text{Var}(Y | X)]$$

For a Poisson law the variance is equal to the mean, $\text{Var}(Y | X = x) = \Phi x$, so

$$R^* = \mathbb{E}_X [\Phi X] = \Phi \mathbb{E}[X] = 50 \times 1.25 = \boxed{62.5 \text{ counts}^2}$$

Question 2 (C)

We take a biased estimator and check that it is worse than the Bayes predictor f^* , in three parts:

- **Estimator.** We pick a biased estimator $\tilde{f}$.
- **True risk (theory).** As a reference value, we compute its true risk $R(\tilde{f})$.
- **Simulation.** We approximate the true risks $R(f^*)$ and $R(\tilde{f})$ by their empirical risk on a test set, and check that the test risk of f^* is below that of $\tilde{f}$ and close to the Bayes risk.

Estimator: We use the biased estimator $\tilde{f}(x) = ax$ with $a = 40 \neq \Phi = 50$. It is biased because it underestimates the activity of the source.

True risk (theory): We follow the course section 3.1.5 (*Bayes estimator for regression and squared loss*).

- **Step 1.** By the definition of the risk (eq. (33), p. 18) and the law of total expectation (eq. (37), p. 18), with ℓ the squared loss and $\tilde{f}(X) = aX$,

$$R(\tilde{f}) = \mathbb{E}_{(X,Y) \sim \rho} [\ell(Y, \tilde{f}(X))] = \mathbb{E}_X \left[\mathbb{E}[(Y - aX)^2 | X] \right]$$

- **Step 2.** We split the error around the conditional mean ΦX , writing $Y - aX = (Y - \Phi X) + (\Phi X - aX)$, and expand the square:

$$\mathbb{E}[(Y - aX)^2 | X] = \underbrace{\mathbb{E}[(Y - \Phi X)^2 | X]}_{(i)} + \underbrace{2(\Phi X - aX) \mathbb{E}[Y - \Phi X | X]}_{(ii)} + (\Phi X - aX)^2$$

The cross term (ii) vanishes since $\mathbb{E}[Y - \Phi X | X] = \mathbb{E}[Y | X] - \Phi X = 0$ (using $\mathbb{E}[Y | X] = \Phi X$ from Q1), and term (i) is the conditional variance $\text{Var}(Y | X)$. Hence

$$\mathbb{E}[(Y - aX)^2 | X] = \text{Var}(Y | X) + (\Phi - a)^2 X^2$$

- **Step 3.** Taking $\mathbb{E}_X$,

$$R(\tilde{f}) = \underbrace{\mathbb{E}_X [\text{Var}(Y | X)]}_{= R^* \text{ (Bayes risk, Q1)}} + (\Phi - a)^2 \mathbb{E}[X^2]$$

The risk splits into the irreducible Bayes risk R^* plus a non-negative bias term coming from the wrong slope $a \neq \Phi$, so $R(\tilde{f}) \geq R^*$, with equality only if $a = \Phi$.

Numerical value: $X \sim \mathcal{U}([0.5, 2.0])$ has $\mathbb{E}[X] = \frac{0.5+2.0}{2} = 1.25$ and $\text{Var}(X) = \frac{(2.0-0.5)^2}{12} = 0.1875$. Using $\mathbb{E}[X^2] = \text{Var}(X) + \mathbb{E}[X]^2$,

$$\mathbb{E}[X^2] = 0.1875 + 1.25^2 = 0.1875 + 1.5625 = 1.75$$

With $R^* = 62.5$ and $(\Phi - a)^2 = (50 - 40)^2 = 100$,

$$R(\tilde{f}) = 62.5 + 100 \times 1.75 = \boxed{237.5 \text{ counts}^2}$$

Simulation: We estimate the true risk of f^* and $\tilde{f}$ by their empirical risk on a large test set (10^6 samples). The simulation gives

$$\hat{R}_{\text{test}}(f^*) = 62.54 \quad \hat{R}_{\text{test}}(\tilde{f}) = 237.49$$

The theoretical values are 62.5 and 237.5, so they match. As expected, $\hat{R}_{\text{test}}(f^*) < \hat{R}_{\text{test}}(\tilde{f})$, and the test error of f^* is almost equal to the Bayes risk. Figure 1 shows that both test risks converge to their true values when the test set grows.

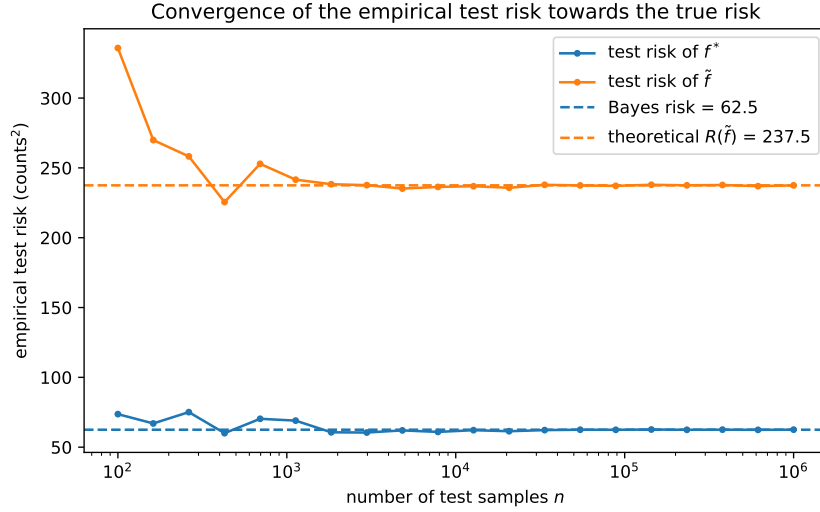


Figure 1: Empirical test risk of f^* and $\tilde{f}$ versus the number of test samples. Both converge to their theoretical values; f^* attains the Bayes risk 62.5.

2 Bayes risk with the absolute loss

Code: `exercise_2_absolute_loss/absolute_loss_simulation.py`.

Question 0 (M)

An example is $f(x) = x^3$ at $x_0 = 0$. We have $f'(x) = 3x^2$, so $f'(0) = 0$. But f is increasing everywhere, so the curve just flattens at x_0 and keeps rising: it is an inflection point, not a minimum or a maximum. So a zero derivative is necessary but not sufficient for a minimum, which is why, in Question 2, we also check the convexity of g before concluding that the critical point is a minimum.

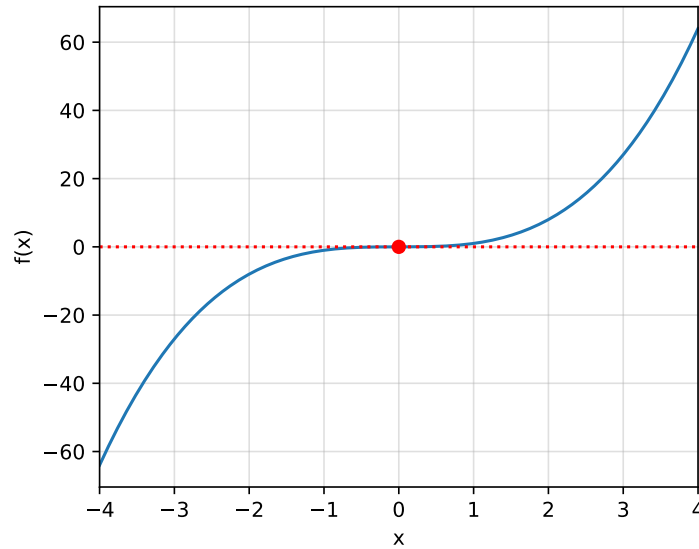


Figure 2: $f(x) = x^3$ flattens at $x_0 = 0$ (horizontal tangent, in red, since $f'(0) = 0$) but keeps increasing through it: x_0 is an inflection point, not an extremum.

Question 1 (M+C)

We check whether the Bayes predictor for the squared loss is also the Bayes predictor for the absolute loss, on a concrete setting, both by hand and by simulation.

Setting: We take $X \sim \mathcal{U}([1, 2])$ and $Y | X = x \sim \text{Exponential}(\lambda = x)$, where λ is the rate. We choose the exponential law because it is asymmetric. For an exponential law of rate λ (with $y \geq 0$):

$$p(y) = \lambda e^{-\lambda y}, \quad F(y) = 1 - e^{-\lambda y}, \quad \mathbb{E}[Y] = \frac{1}{\lambda}$$

Candidates: Our goal is to show that the absolute-loss Bayes predictor is not the same as the squared-loss one. We know the squared-loss Bayes predictor: it is the conditional mean (eq. (41), p. 20), $f_{\text{sq}}^* = m$. So it is enough to find any predictor with a *smaller* absolute-loss risk than m : if the mean can be beaten under the absolute loss, then the mean is not the absolute-loss Bayes predictor.

A natural challenger is the conditional median q , because the exponential law is asymmetric, so its mean and median differ ($m \neq q$). We therefore compare the two candidates, the conditional mean m and the conditional median q , on their absolute-loss risk, by computation and by simulation.

Computing the two candidates: For an exponential law of rate λ , the mean is $\mathbb{E}[Y] = 1/\lambda$. The median q is the value that splits the law into two equal parts, so it satisfies

$$P(Y \leq q) = F(q) = \frac{1}{2}$$

Solving this equation:

$$1 - e^{-\lambda q} = \frac{1}{2} \iff e^{-\lambda q} = \frac{1}{2} \iff -\lambda q = \ln \frac{1}{2} \iff q = \frac{\ln 2}{\lambda}$$

With $\lambda = x$:

$$m(x) = \mathbb{E}[Y \mid X = x] = \frac{1}{x}, \quad q(x) = \text{median}(Y \mid X = x) = \frac{\ln 2}{x}$$

As announced, the mean and median differ: $m(x) \neq q(x)$.

Comparing the candidates by computation: We now compute the absolute-loss risk of each candidate. Starting from the definition of the risk (eq. (33), p. 18) and applying the law of total expectation (eq. (37), p. 18),

$$R_{\ell_{\text{abs}}}(f) = \mathbb{E}_{(X,Y) \sim \rho}[\ell_{\text{abs}}(Y, f(X))] = \mathbb{E}_{(X,Y) \sim \rho}[|Y - f(X)|] = \mathbb{E}_X[\mathbb{E}[|Y - f(X)| \mid X]]$$

so the computation has two steps: the inner conditional risk at a fixed x , then the average over X .

- **Inner conditional risk (fixed x).** At a fixed x , $f(x)$ is a constant z , and in our setting $Y \mid X = x$ has density $x e^{-xy}$ (rate x). The conditional risk is then

$$\mathbb{E}[|Y - z| \mid X = x] = \int_0^\infty |y - z| x e^{-xy} dy = \int_0^z (z - y) x e^{-xy} dy + \int_z^\infty (y - z) x e^{-xy} dy$$

- *First piece.* We split it in two:

$$\int_0^z (z - y) x e^{-xy} dy = z \int_0^z x e^{-xy} dy - \int_0^z y x e^{-xy} dy$$

- *First term:* the antiderivative of $x e^{-xy}$ is $-e^{-xy}$, so

$$z \int_0^z x e^{-xy} dy = z [-e^{-xy}]_0^z = z(1 - e^{-xz})$$

- *Second term:* by integration by parts ($u = y$, $du = dy$, $dv = x e^{-xy} dy$, $v = -e^{-xy}$)

$$\int_0^z y x e^{-xy} dy = [-y e^{-xy}]_0^z + \int_0^z e^{-xy} dy = -z e^{-xz} + [-\frac{1}{x} e^{-xy}]_0^z = -z e^{-xz} + \frac{1}{x}(1 - e^{-xz})$$

Hence the first piece equals

$$\int_0^z (z - y) x e^{-xy} dy = z(1 - e^{-xz}) - \left(-z e^{-xz} + \frac{1}{x}(1 - e^{-xz})\right) = z - \frac{1}{x} + \frac{1}{x} e^{-xz}$$

- *Second piece.* We substitute $t = y - z$, so $dt = dy$, the bounds $y : z \rightarrow \infty$ become $t : 0 \rightarrow \infty$, and $e^{-xy} = e^{-x(t+z)} = e^{-xz} e^{-xt}$. This gives

$$\int_z^\infty (y - z) x e^{-xy} dy = e^{-xz} \int_0^\infty t x e^{-xt} dt$$

The remaining integral is computed by integration by parts ($u = t$, $du = dt$, $dv = x e^{-xt} dt$, $v = -e^{-xt}$):

$$\int_0^\infty t x e^{-xt} dt = [-t e^{-xt}]_0^\infty + \int_0^\infty e^{-xt} dt = 0 + [-\frac{1}{x} e^{-xt}]_0^\infty = \frac{1}{x}$$

Hence the second piece equals $e^{-xz} \cdot \frac{1}{x} = \frac{1}{x} e^{-xz}$.

◦ *Sum.* Adding the two pieces,

$$\mathbb{E}[|Y - z| \mid X = x] = \left(z - \frac{1}{x} + \frac{1}{x} e^{-xz} \right) + \frac{1}{x} e^{-xz} = z - \frac{1}{x} + \frac{2}{x} e^{-xz}$$

For our two candidates:

$$\text{median } z = q(x) = \frac{\ln 2}{x} : \quad \mathbb{E}[|Y - q(x)| \mid X = x] = \frac{\ln 2}{x} - \frac{1}{x} + \frac{2}{x} \cdot \frac{1}{2} = \frac{\ln 2}{x}$$

$$\text{mean } z = m(x) = \frac{1}{x} : \quad \mathbb{E}[|Y - m(x)| \mid X = x] = \frac{1}{x} - \frac{1}{x} + \frac{2}{x} \cdot \frac{1}{e} = \frac{2}{ex}$$

- **Average over X .** The true risk averages these conditional risks over $X \sim \mathcal{U}([1, 2])$. We first compute the expectation

$$\mathbb{E}\left[\frac{1}{X}\right] = \int_1^2 \frac{1}{x} dx = [\ln x]_1^2 = \ln 2$$

so

$$R_{\ell_{\text{abs}}}(q) = \mathbb{E}_X\left[\frac{\ln 2}{X}\right] = \ln 2 \mathbb{E}\left[\frac{1}{X}\right] = (\ln 2)^2 \approx 0.4805$$

$$R_{\ell_{\text{abs}}}(m) = \mathbb{E}_X\left[\frac{2}{eX}\right] = \frac{2}{e} \mathbb{E}\left[\frac{1}{X}\right] = \frac{2}{e} \ln 2 \approx 0.5100$$

So $R_{\ell_{\text{abs}}}(q) < R_{\ell_{\text{abs}}}(m)$: the median beats the mean under the absolute loss. The mean is therefore not the absolute-loss Bayes predictor, so it differs from the squared-loss one: $f_{\ell_{\text{abs}}}^* \neq f_{\ell_{\text{sq}}}^*$.

Simulation: A simulation on 10^6 test samples confirms this, and also the opposite for the squared loss (each loss is smallest for its own Bayes predictor, in bold):

predictor	test $R_{\ell_{\text{abs}}}$	test $R_{\ell_{\text{sq}}}$
median $q(x) = \ln 2/x$	0.4799	0.5464
mean $m(x) = 1/x$	0.5094	0.4994

Figure 3 shows the two objectives $z \mapsto \mathbb{E}[|Y - z|]$ and $z \mapsto \mathbb{E}[(Y - z)^2]$ for a fixed x : the first is minimal at the median, the second at the mean.

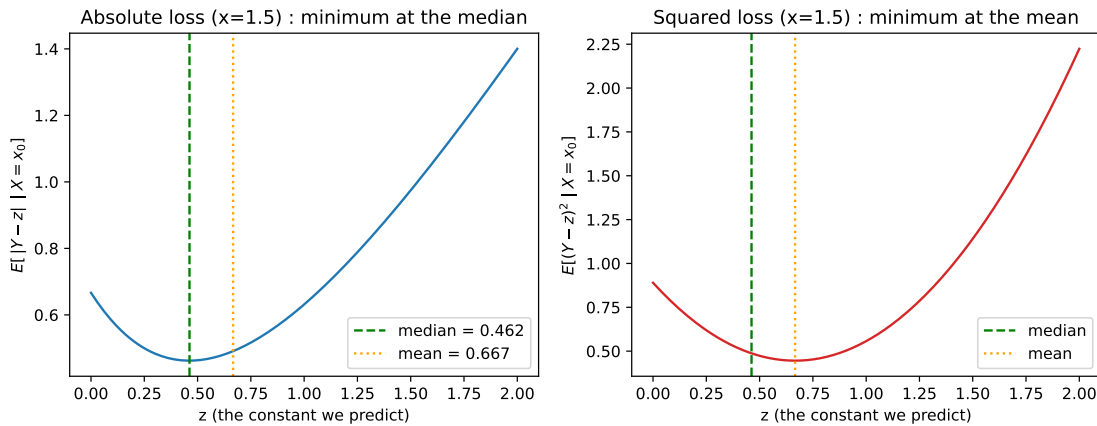


Figure 3: For a fixed $x = 1.5$: the absolute-loss objective (left) is minimised at the conditional median, the squared-loss objective (right) at the conditional mean.

Question 2 (M)

Fix x . Let p be the conditional density of $Y \mid X = x$ and F its cumulative distribution function. We assume $Y \mid X = x$ has a finite first moment, that is $\mathbb{E}[|Y| \mid X = x] < \infty$. As noted in the statement, this implies that for every $z \in \mathbb{R}$ the variable $Y - z \mid X = x$ also has a finite first moment, so the function we study,

$$g(z) = \mathbb{E}[|Y - z| \mid X = x] = \int_{\mathbb{R}} |y - z| p(y) dy = \int_{-\infty}^z (z - y) p(y) dy + \int_z^{+\infty} (y - z) p(y) dy$$

is well-defined and finite for every z . We now compute its derivative and show it is convex.

Derivative: We use the Leibniz rule, which differentiates an integral whose bounds *and* integrand depend on z :

$$\frac{d}{dz} \int_{a(z)}^{b(z)} h(z, y) dy = h(z, b(z)) b'(z) - h(z, a(z)) a'(z) + \int_{a(z)}^{b(z)} \frac{\partial}{\partial z} h(z, y) dy$$

We apply it to the two pieces of g .

- *First integral* $\int_{-\infty}^z (z - y)p(y) dy$. Here $h(z, y) = (z - y)p(y)$, the lower bound $a(z) = -\infty$ is fixed ($a'(z) = 0$), and the upper bound $b(z) = z$ moves ($b'(z) = 1$). The three Leibniz terms are

$$\begin{aligned} \frac{d}{dz} \int_{-\infty}^z (z - y)p(y) dy &= \underbrace{(z - z)p(z) \cdot 1}_{\text{upper bound}=0} - \underbrace{0}_{\text{lower bound fixed}} + \int_{-\infty}^z \underbrace{\frac{\partial}{\partial z} [(z - y)p(y)]}_{=p(y)} dy \\ &= \int_{-\infty}^z p(y) dy = F(z) \end{aligned}$$

- *Second integral* $\int_z^{+\infty} (y - z)p(y) dy$. Here $h(z, y) = (y - z)p(y)$, the upper bound $b(z) = +\infty$ is fixed ($b'(z) = 0$), and the lower bound $a(z) = z$ moves ($a'(z) = 1$). The three Leibniz terms are

$$\begin{aligned} \frac{d}{dz} \int_z^{+\infty} (y - z)p(y) dy &= \underbrace{0}_{\text{upper bound fixed}} - \underbrace{(z - z)p(z) \cdot 1}_{\text{lower bound}=0} + \int_z^{+\infty} \underbrace{\frac{\partial}{\partial z} [(y - z)p(y)]}_{=-p(y)} dy \\ &= - \int_z^{+\infty} p(y) dy = -(1 - F(z)) \end{aligned}$$

Adding the two,

$$g'(z) = F(z) - (1 - F(z)) = 2F(z) - 1$$

Minimiser: F is a cumulative distribution function, hence non-decreasing, so $g'(z) = 2F(z) - 1$ is non-decreasing too, which means g is convex. For a convex function, any point where g' is zero is automatically a global minimum, so it suffices to solve $g'(z) = 0$:

$$g'(z) = 0 \iff 2F(z) - 1 = 0 \iff F(z) = \frac{1}{2}$$

A value z with $F(z) = \frac{1}{2}$ is, by definition, a *median* of $Y \mid X = x$, so the minimiser of g is exactly the conditional median. Therefore

$$\boxed{f_{\ell_{\text{abs}}}^*(x) = \arg \min_{z \in \mathbb{R}} \mathbb{E}[|Y - z| \mid X = x] = \text{median}(Y \mid X = x).}$$

This is the result we used in Question 1.

3 Expected value of the empirical risk for OLS

Code: `exercise_3_ols_risk/ols_sigma_simulation.py`.

Throughout, the model is linear with fixed design, $y = X\theta^* + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2 I_n)$, $X \in \mathbb{R}^{n \times d}$ fixed and injective ($d \leq n$), and $\hat{\theta} = (X^\top X)^{-1} X^\top y$. The empirical risk is $R_n(\theta) = \frac{1}{n} \|y - X\theta\|^2$. The goal of the exercise is to prove the proposition below, then use and discuss it in the following questions.

Proposition 1. *In the linear model with fixed design, $\mathbb{E}[R_n(\hat{\theta})] = \frac{n-d}{n} \sigma^2$*

Note: the risk in Proposition 1 is a *training* error: $\hat{\theta}$ is computed from the training data, and $R_n(\hat{\theta})$ is its error on that same training set.

Question 1 (M)

We compare the value of Proposition 1 (the expected empirical risk of OLS) with the Bayes risk, say which one is smaller, and interpret the gap as a function of n and d .

Bayes risk:

- **Bayes predictor.** For the squared loss it is the conditional mean (eq. (41), p. 20); from the model $Y = X^\top \theta^* + \epsilon$, at $X = x$:

$$f^*(x) = \mathbb{E}[Y \mid X = x] = \mathbb{E}[x^\top \theta^* + \epsilon \mid X = x] = x^\top \theta^*$$

because $x^\top \theta^*$ is constant given $X = x$, and $\mathbb{E}[\epsilon \mid X = x] = \mathbb{E}[\epsilon] = 0$ (ϵ independent of X , centred).

- **Bayes risk.** It follows from its definition (eq. (40), p. 19), here with the squared loss, the infimum being attained at the conditional mean f^* above:

$$R^* = \mathbb{E}_X \left[\inf_{y \in \mathbb{R}} \mathbb{E}_Y [(Y - y)^2 \mid X] \right] = \mathbb{E}_X \left[\mathbb{E}_Y [(Y - f^*(X))^2 \mid X] \right] = \mathbb{E}[(Y - X\theta^*)^2] = \mathbb{E}[\epsilon^2]$$

using $f^*(X) = X\theta^*$ and $Y - X\theta^* = \epsilon$. Hence

$$R^* = \text{Var}(\epsilon) + \mathbb{E}[\epsilon]^2 = \text{Var}(\epsilon) = \sigma^2$$

This Bayes risk is irreducible: ϵ is independent of X , so no predictor can drive the error below σ^2 .

Comparison: Proposition 1 gives the expected empirical risk of OLS:

$$\mathbb{E}[R_n(\hat{\theta})] = \frac{n-d}{n} \sigma^2$$

Since $\frac{n-d}{n} < 1$, it is *smaller* than the Bayes risk, by exactly

$$\mathbb{E}[R_n(\hat{\theta})] = \frac{n-d}{n} \sigma^2 < \sigma^2 = R^*, \quad R^* - \mathbb{E}[R_n(\hat{\theta})] = \frac{d}{n} \sigma^2$$

Interpretation: At first sight this looks impossible: how can OLS beat the Bayes risk, which is the best possible? The reason is that the two quantities are not the same kind of error:

- the Bayes risk σ^2 is a *true* risk, the error on *new* data, and no predictor can go below it there;
- $\frac{n-d}{n} \sigma^2$ is the *empirical* risk $R_n(\hat{\theta}) = \frac{1}{n} \|y - X\hat{\theta}\|^2$, measured on the *same* data that produced $\hat{\theta}$.

So there is no contradiction. The training risk falls below σ^2 because of overfitting: θ^* would give a training error of σ^2 on average, but OLS instead picks $\hat{\theta}$ to minimise the training error, so $\hat{\theta}$ fits part of the noise ϵ .

Discussion in n and d :

- **Large n (fixed d):** $\frac{n-d}{n} \rightarrow 1$, so the training risk returns to σ^2 and the gap $\frac{d}{n}\sigma^2 \rightarrow 0$, so more data means less overfitting.
- **d close to n :** $\frac{n-d}{n} \rightarrow 0$, the training risk collapses to 0: the model tends toward overfitting, interpolating the data exactly.

On *new* data, OLS can never beat the Bayes risk σ^2 . In fact, it does worse: the same overfitting that pushed the training error below σ^2 pushes the true risk on new data *above* σ^2 . So OLS only looked better than the Bayes predictor on the training set. On new data, it is actually worse.

Question 2 (M)

With $\hat{\theta} = (X^\top X)^{-1} X^\top y$, the prediction error $y - X\hat{\theta}$ is

$$y - X\hat{\theta} = y - X(X^\top X)^{-1} X^\top y = (I_n - X(X^\top X)^{-1} X^\top) y$$

We now replace y by $X\theta^* + \epsilon$. Note that

$$(I_n - X(X^\top X)^{-1} X^\top) X = X - X(X^\top X)^{-1} X^\top X = X - X[(X^\top X)^{-1} (X^\top X)] = X - X I_d = 0$$

So the deterministic part vanishes and only the noise survives:

$$y - X\hat{\theta} = (I_n - X(X^\top X)^{-1} X^\top) (X\theta^* + \epsilon) = (I_n - X(X^\top X)^{-1} X^\top) \epsilon$$

By definition, $R_n(\hat{\theta}) = \frac{1}{n} \|y - X\hat{\theta}\|^2$. Taking the expectation over ϵ gives

$$\mathbb{E}[R_n(\hat{\theta})] = \mathbb{E}_\epsilon \left[\frac{1}{n} \|(I_n - X(X^\top X)^{-1} X^\top) \epsilon\|^2 \right]$$

Question 3 (M)

For $A \in \mathbb{R}^{n \times n}$, the (j, j) entry of $A^\top A$ is

$$(A^\top A)_{jj} = \sum_{i=1}^n (A^\top)_{ji} A_{ij} = \sum_{i=1}^n A_{ij}^2$$

Summing over j ,

$$\text{tr}(A^\top A) = \sum_{j=1}^n (A^\top A)_{jj} = \sum_{j=1}^n \sum_{i=1}^n A_{ij}^2 = \sum_{(i,j) \in [1,n]^2} A_{ij}^2$$

Question 4 (M)

$$\|A\epsilon\|^2 = (A\epsilon)^\top (A\epsilon) = \epsilon^\top A^\top A \epsilon = \sum_{i,j} (A^\top A)_{ij} \epsilon_i \epsilon_j$$

The noise is independent and centred, so for $i \neq j$, $\mathbb{E}[\epsilon_i \epsilon_j] = \mathbb{E}[\epsilon_i] \mathbb{E}[\epsilon_j] = 0 \times 0 = 0$. For $i = j$, since $\mathbb{E}[\epsilon_i] = 0$, $\mathbb{E}[\epsilon_i^2] = \text{Var}(\epsilon_i) + \mathbb{E}[\epsilon_i]^2 = \text{Var}(\epsilon_i) = \sigma^2$. Only the diagonal terms $i = j$ survive in the sum, so

$$\mathbb{E}_\epsilon \|A\epsilon\|^2 = \sum_{i,j} (A^\top A)_{ij} \mathbb{E}[\epsilon_i \epsilon_j] = \sum_i (A^\top A)_{ii} \sigma^2 = \sigma^2 \text{tr}(A^\top A)$$

therefore $\mathbb{E}_\epsilon \left[\frac{1}{n} \|A\epsilon\|^2 \right] = \frac{\sigma^2}{n} \text{tr}(A^\top A)$.

Question 5 (M)

Let $M := X(X^\top X)^{-1}X^\top$, so that $A = I_n - M$. We show $A^\top A = A$ in four steps.

Step 1: show that M is symmetric: $X^\top X$ is symmetric by construction. The inverse of a symmetric matrix is symmetric, so $(X^\top X)^{-1}$ is symmetric too, that is $[(X^\top X)^{-1}]^\top = (X^\top X)^{-1}$. Hence

$$M^\top = (X(X^\top X)^{-1}X^\top)^\top = X[(X^\top X)^{-1}]^\top X^\top = X(X^\top X)^{-1}X^\top = M$$

Step 2: show that A is symmetric: Using Step 1, $M^\top = M$:

$$A^\top = (I_n - M)^\top = I_n^\top - M^\top = I_n - M = A$$

Step 3: show that M is idempotent ($M^2 = M$):

$$\begin{aligned} M^2 &= X(X^\top X)^{-1}X^\top X(X^\top X)^{-1}X^\top = X[(X^\top X)^{-1}(X^\top X)](X^\top X)^{-1}X^\top \\ &= X(X^\top X)^{-1}X^\top = M \end{aligned}$$

Step 4: show that A is idempotent ($A^2 = A$): Using Step 3, $M^2 = M$:

$$A^2 = (I_n - M)^2 = I_n - 2M + M^2 = I_n - 2M + M = I_n - M = A$$

Conclusion: Using Step 2, $A^\top = A$, so $A^2 = AA = A^\top A$. And Step 4 shows $A^2 = A$. So $A^\top A = A$.

Question 6 (M)

From Question 2, $y - X\hat{\theta} = A\epsilon$. From Question 4, $\mathbb{E}_\epsilon[\frac{1}{n}\|A\epsilon\|^2] = \frac{\sigma^2}{n} \text{tr}(A^\top A)$. And from Question 5, $A^\top A = A$. Combining the three,

$$\mathbb{E}[R_n(\hat{\theta})] = \mathbb{E}_\epsilon\left[\frac{1}{n}\|A\epsilon\|^2\right] = \frac{\sigma^2}{n} \text{tr}(A^\top A) = \frac{\sigma^2}{n} \text{tr}(A)$$

With $M = X(X^\top X)^{-1}X^\top$, so that $A = I_n - M$,

$$\text{tr}(A) = \text{tr}(I_n - M) = \text{tr}(I_n) - \text{tr}(M) = n - \text{tr}(M)$$

By the cyclic property of the trace,

$$\text{tr}(M) = \text{tr}(X(X^\top X)^{-1}X^\top) = \text{tr}((X^\top X)^{-1}X^\top X) = \text{tr}(I_d) = d$$

so $\text{tr}(A) = n - d$, and

$$\mathbb{E}[R_n(\hat{\theta})] = \frac{n-d}{n}\sigma^2$$

which proves Proposition 1.

Question 7 (M)

By definition $R_n(\hat{\theta}) = \frac{1}{n}\|y - X\hat{\theta}\|_2^2$, so $\|y - X\hat{\theta}\|_2^2 = n R_n(\hat{\theta})$. Proposition 1 then gives

$$\mathbb{E}[\|y - X\hat{\theta}\|_2^2] = n \mathbb{E}[R_n(\hat{\theta})] = n \cdot \frac{n-d}{n}\sigma^2 = (n-d)\sigma^2$$

Since $n - d$ is a constant,

$$\mathbb{E}\left[\frac{\|y - X\hat{\theta}\|_2^2}{n-d}\right] = \frac{\mathbb{E}[\|y - X\hat{\theta}\|_2^2]}{n-d} = \frac{(n-d)\sigma^2}{n-d} = \sigma^2$$

Question 8 (C)

This question checks Proposition 1 and Question 7 numerically, by simulation.

Setup: We fix one design X with $n = 200$ samples and $d = 40$ features, and one parameter θ^* to generate the data. We also fix the true noise variance to $\sigma^2 = 4$ (so $\sigma = 2$).

Simulation: We repeat the following 20,000 times. Draw a fresh noise $\epsilon \sim \mathcal{N}(0, \sigma^2 I_n)$ and build $y = X\theta^* + \epsilon$. Fit OLS to get $\hat{\theta}$, then record two numbers:

- the empirical risk $R_n(\hat{\theta})$, whose average should match Proposition 1: $\frac{n-d}{n}\sigma^2$;
- the variance estimator $\hat{\sigma}^2 := \|y - X\hat{\theta}\|_2^2/(n-d)$ from Question 7, whose average should match the true σ^2 .

Result: Averaging over the 20,000 repetitions,

$$\text{mean of } R_n(\hat{\theta}) = 3.198 \quad (\text{theory } \frac{n-d}{n}\sigma^2 = 3.2), \quad \text{mean of } \hat{\sigma}^2 = 3.998 \quad (\text{true } \sigma^2 = 4)$$

Both recorded values match the theory closely. Figure 4 shows how $\hat{\sigma}^2$ converges to σ^2 as the number of repetitions grows.

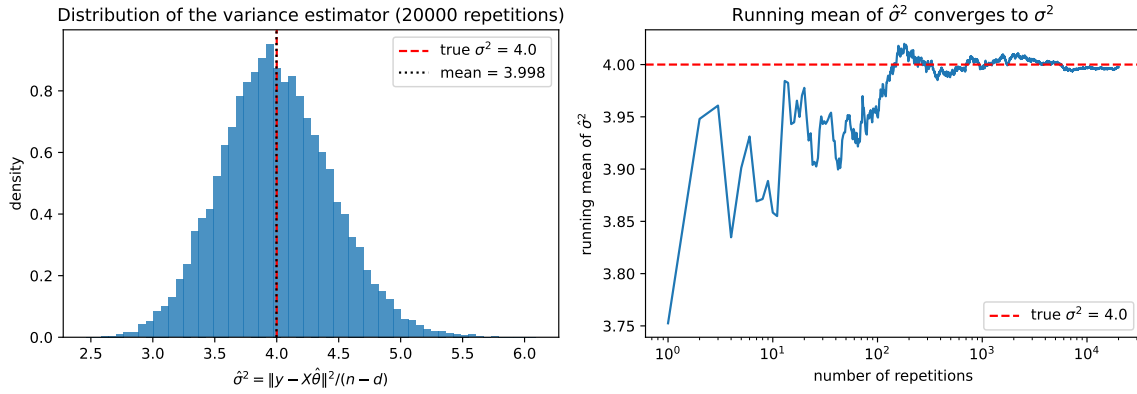


Figure 4: Left: distribution of $\hat{\sigma}^2 = \|y - X\hat{\theta}\|_2^2/(n-d)$ over 20,000 repetitions, centred on the true $\sigma^2 = 4$. Right: the running mean of $\hat{\sigma}^2$ converges to σ^2 .

4 Regression on a given dataset

Code: `exercise_4_regression/regression.py`.

Setting and difficulty: The dataset has $n_{\text{train}} = 200$ samples, $n_{\text{test}} = 200$ samples, and $d = 200$ features. The features already share the same scale, between 0 and 1, so we do not standardize them before regularization. With as many parameters as samples, unregularized OLS overfits: it fits the noise along with the true signal, so we need regularization. We compare four linear estimators. Each regularized model is tuned by a single 5-fold cross-validated grid search on the training set only, which selects the hyperparameter and gives the CV score. The test set is scored once, at the end.

Models and tuning: Hyperparameters for each model are chosen by the grid search described above.

- **OLS** (LinearRegression): no regularization, our baseline.
- **Ridge** (L2): α searched over $[10^{-3}, 10^3]$.
- **Lasso** (L1): α searched over $[10^{-4}, 10]$.
- **ElasticNet** (L1+L2): same α range as Lasso, with `l1_ratio` $\in \{0.1, 0.3, 0.5, 0.7, 0.9\}$.

Results:

model	CV R^2	train R^2	test R^2	non-zero	selected hyper-parameters
OLS	0.508	1.000	−9.919	200	none
Ridge	0.624	0.985	0.710	200	$\alpha = 1.23$
Lasso	0.936	0.966	0.923	46	$\alpha = 4.6 \times 10^{-3}$
ElasticNet	0.934	0.965	0.921	46	$\alpha = 5.2 \times 10^{-3}$, <code>l1_ratio</code> = 0.9

Discussion: OLS, with no regularization at all, overfits completely: it reaches a train R^2 of 1 but collapses to -9.919 on the test set. Lasso reaches a **test** $R^2 = 0.923$, above the target 0.88 and close to the announced Bayes level of ≈ 0.92 (Figure 5, right). ElasticNet comes just behind at 0.921, and Ridge is far below both, at 0.71.

The reason is visible in the solution: Lasso keeps only 46 of the 200 coefficients, so only a few features actually matter. Ridge, on the other hand, spreads its penalty over all coordinates and never sets any of them to zero.

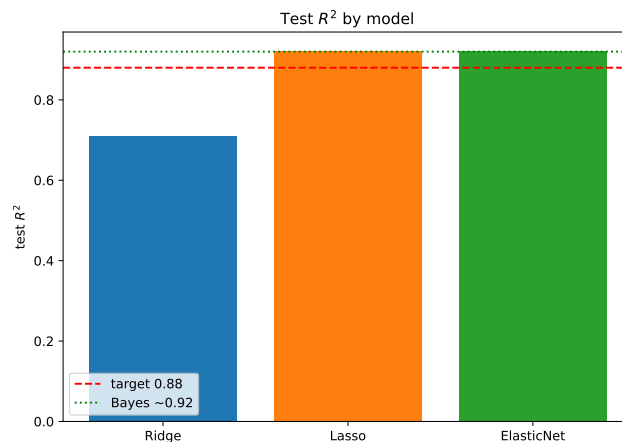


Figure 5: Test R^2 of the regularised models against the 0.88 target and the ≈ 0.92 Bayes level (OLS is excluded, its test R^2 is -9.9).

5 Classification on a given dataset

Code: `exercise_5_classification/classification.py`.

Setting: This is a binary, balanced classification problem, with $n_{\text{train}} = 2000$, $n_{\text{test}} = 2000$, and $d = 30$ continuous features (class counts 1019/981 in the training set). For each model, hyperparameters are selected by 5-fold stratified cross-validation on the training set only, using `GridSearchCV`.

The sections below follow the investigation in the order it happened: first a standardised baseline, which misses the target, then a diagnostic to understand why, then a test of the resulting hypothesis, and finally the final model.

Step 1: baseline, everything standardised: Standardising is the usual best practice, so this is tried first, for all five classifiers suggested in the statement (both the RBF and the polynomial kernel for the SVC), each tuned by the 5-fold CV described above.

model (standardised)	CV accuracy	test accuracy
SVC (RBF), $C=5, \gamma=0.08$	0.773	0.799
k -NN ($k=27$, distance, $p=1$)	0.754	0.784
SVC (poly kernel), degree 3, $C=3, \gamma=0.02$	0.741	0.782
MLP (100, $\alpha=1$)	0.743	0.769
AdaBoost (500 stumps, $\text{lr}=0.5$)	0.718	0.751
Logistic Regression (linear, $C=0.1$)	0.709	0.744

None of the six reaches the 0.85 target. All test accuracies stay close together, between 0.744 and 0.799, no matter the type of model. The best is the RBF kernel, at 0.799.

Step 2: diagnostic: Why does every standardised model stay stuck around 0.75–0.80? For each of the 30 features, we compute two numbers: how informative it is on its own (its mutual information with the label), and how much its values vary (its standard deviation). Figure 6 plots one point per feature, with mutual information on one axis and standard deviation on the other.

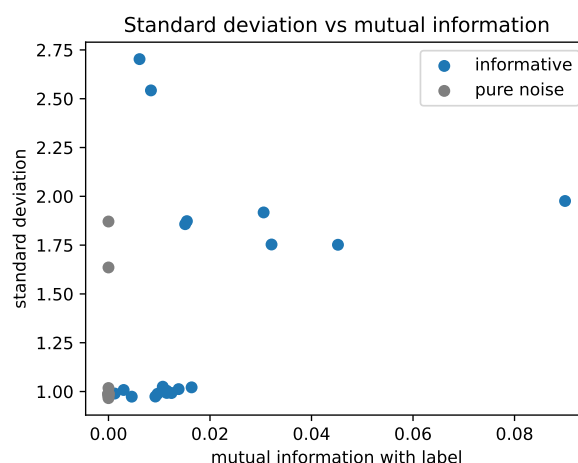


Figure 6: For each of the 30 features: mutual information with the label vs standard deviation on the training set

On average, informative features carry more variance than noise features, and standardising removes that difference. So standardising puts noise features on the same level as informative ones, which penalizes models that rely on feature scale.

Step 3: hypothesis test: To test this, we now try every model again without the scaler (except AdaBoost, which is not affected by scaling).

model (raw features)	CV accuracy	test accuracy
SVC (poly kernel), degree 3, $C=3$, $\gamma=\text{scale}$	0.764	0.899
SVC (RBF), $C=1$, $\gamma=0.03$	0.798	0.805
k -NN, $k=21$, uniform, $p=2$	0.782	0.786
MLP, (100, 50), $\alpha=1$	0.758	0.782
Logistic Regression, $C=0.001$	0.716	0.738

Only the polynomial-kernel SVC really improves without the scaler, from 0.782 to **0.899**. The other models barely move. The reason is that the polynomial kernel compares two points with a dot product, and in a dot product, features with more variance naturally count more. Since the informative features already have more variance than the noise features (Step 2), keeping the data raw lets the kernel favour the informative features automatically. Scaling removes this advantage by giving every feature the same variance.

Limits of the selection protocol: One point to note: using cross-validation alone, we would not have picked our final model. On raw features, the RBF kernel has the best CV accuracy (0.798, versus 0.764 for the degree-3 polynomial kernel), but it only reaches 0.805 on the test set, below the 0.85 target. Only the test scores show that the polynomial kernel is better, so the test set did influence our choice. This is exactly the “pretend not to use the test set” situation from the project statement, but it is a deliberate choice: without the 0.85 target, we would have picked the RBF kernel instead.

Conclusion: The final model is `SVC(kernel="poly", degree=3, coef0=0)` on raw features, with $C=3$ and $\gamma=\text{scale}$. It reaches a **test accuracy of 0.899**, above the 0.85 target. Figure 7 compares its test accuracy to every model tried in Steps 1 and 3, scaled and raw.

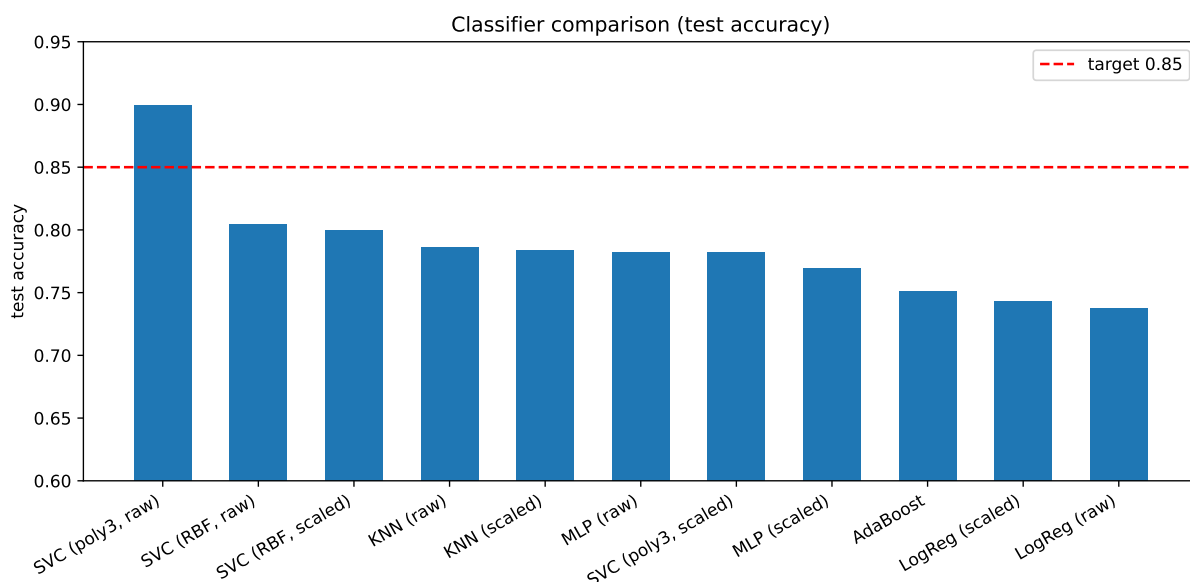


Figure 7: Test accuracy of every model tried, scaled and raw, sorted best to worst. Only the degree-3 polynomial kernel SVC on raw features clears the 0.85 target (red), at 0.899.

6 Application of supervised learning: credit-card fraud detection

Code: `exercise_6_supervised/creditcard_fraud.py`, data `data/creditcard.csv.gz`.

Dataset and problem: We first chose the *Fraud Detection* dataset (<https://www.kaggle.com/datasets/waddahali/fraud-detection>), but its classes were too poorly distributed to study, so we changed to the *Credit Card Fraud Detection* dataset (<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>). It contains real card payments made by European cardholders over two days in September 2013. The full file has 284,807 payments. We keep a smaller random sample of 120,000 payments, with the same fraud rate. We want to predict the target **Class** (1 = fraud, 0 = normal) from the payment features:

- **Time:** number of seconds between the payment and the first payment in the data;
- **Amount:** the amount of the payment;
- **V1–V28:** 28 features that the data provider built by running a PCA on the original variables (to hide them and protect cardholder confidentiality). So we only know these 28 features are decorrelated and centred, with no clear meaning.

Fraud is very rare (0.17% of payments), so the problem is strongly imbalanced: accuracy is useless here, since always predicting “normal” already scores 99.83%. And the two types of error do not cost the same: a missed fraud is a direct loss, and a false alarm bothers a normal customer. So what matters is the balance between precision and recall.

Methodology: We split the data 75/25 into train and test, balanced by class (same fraud rate in both sets). The test set is used only once, at the very end. On the train set, a 3-fold cross-validation first compares six models, each with one fixed, reasonable configuration, then tunes the best one’s hyper-parameters with a grid search.

We use six models: logistic regression, k -nearest neighbours, an SVM with an RBF kernel, a neural network (MLP), random forest, and AdaBoost. Features are standardised beforehand, so **Time** and **Amount** do not dominate the scale-sensitive models.

Since accuracy is useless here, we use precision, recall, F1 and the area under the precision-recall curve (*PR-AUC*) instead. Unlike the others, PR-AUC needs no fixed threshold: it scores how well a model ranks frauds above genuine payments across every threshold, so we use it to select the model and its hyper-parameters. The one remaining choice, the decision threshold, is set by maximising F1.

Results:

model	CV		Test			
	PR-AUC	F1	PR-AUC	precision	recall	F1
Random baseline	0.002	0.00	0.002	0.00	0.00	0.00
Logistic regression	0.775	0.11	0.835	0.06	0.94	0.11
k -nearest neighbours	0.721	0.79	0.823	0.87	0.79	0.83
SVM (RBF kernel)	0.504	0.52	0.652	0.59	0.65	0.62
Neural network (MLP)	0.732	0.76	0.761	0.91	0.77	0.83
Random forest	0.782	0.77	0.906	0.95	0.79	0.86
AdaBoost	0.744	0.75	0.855	0.89	0.77	0.83
<i>Improving the random forest (same model throughout):</i>						
Default (threshold 0.5)	0.782	0.77	0.906	0.95	0.79	0.86
Tuned threshold (0.14)	0.782	0.85	0.906	0.85	0.87	0.86
Grid search + threshold (0.13)	0.801	0.84	0.892	0.82	0.89	0.85

How to read the table:

- **Random baseline:** a dummy model that flags fraud at random, with a probability equal to the fraud rate. It is the reference that every model must beat.
 - **Phase 1, model selection:** the six models are compared by cross-validation on the train set, and we keep the best one on the CV PR-AUC, because, as explained above, it needs no fixed threshold and scores how well a model ranks frauds in general. The random forest wins (0.782).
 - **Phase 2, improving the random forest:** we then try two ways to improve the random forest classifier.
 - *Tuned threshold:* keep the same forest and only change the *decision threshold*. 0.5 is too cautious at 0.17% fraud, so we lower it to the F1-optimal value of 0.14.
 - *Grid search:* search over the forest's hyper-parameters to maximise the CV PR-AUC, then re-tune the threshold (again maximising F1) for that new forest, giving 0.13.
- So the default row uses threshold 0.5, and the other two rows use their own tuned threshold.

Discussion: have we solved the problem? Yes, every model beats the random test PR-AUC of 0.002, and the random forest is the best. Also, we compare three variants of that same forest, default, tuned threshold, and grid search (confusion matrices in Figure 8, the test set has 52 frauds):

- **Default (threshold 0.5):** precision is 0.95 and recall is 0.79. It catches 41 of the 52 frauds (11 missed) for only 2 false alarms. This gives the best precision and the best test PR-AUC (0.906).
- **Tuned threshold (0.14):** same forest, lower threshold. Precision drops to 0.85 and recall rises to 0.87: 45 frauds caught (7 missed) for 8 false alarms. The model itself is unchanged, so PR-AUC stays at 0.906 and F1 is unchanged (0.86).
- **Grid search + threshold (0.13):** the CV PR-AUC rises (0.78 $\rightarrow$ 0.80), but the test PR-AUC *drops* (0.906 $\rightarrow$ 0.892), which can be a sign of overfitting to the CV folds. It reaches the highest recall (0.89, 46 frauds caught) but the lowest F1 (0.85).

So which is best? It depends on what we want:

- **Default vs tuned threshold:** tied on PR-AUC and F1, so it is a precision/recall trade-off, not one beating the other. The default (0.95/0.79) misses more fraud but rarely bothers a genuine customer. The tuned threshold (0.85/0.87) catches more fraud at the cost of a few more false alarms. Since a missed fraud is a direct money loss and a false alarm only costs a quick check, the tuned threshold is the better choice in practice.
- **The grid search:** its CV PR-AUC rises (0.78 $\rightarrow$ 0.80) but its test PR-AUC drops (0.906 $\rightarrow$ 0.892), which can be a sign of overfitting to the CV folds. But it does reach the best recall (0.89, 46 frauds caught), so it is not useless in the case of fraud, where a missed fraud is very costly, even if it is not the most versatile choice.

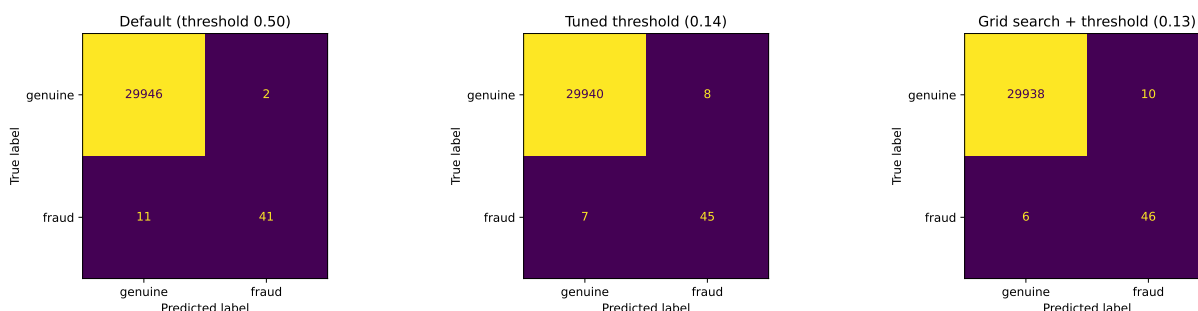


Figure 8: Confusion matrices on the test set for the three random-forest variants.

7 Application of unsupervised learning: Spotify “vibe” clustering

Code: `exercise_7_unsupervised/spotify_clustering.py`, data `data/spotify_tracks.csv`.

Dataset and problem: The *Spotify Tracks Dataset* (<https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>) describes 114,000 tracks. After dropping duplicate `track_ids` we keep 89,741 unique tracks. We cluster them, with k -means, by how they sound (their overall feel, or “vibe”) from their audio features only. This is unsupervised: the provided `track_genre` is never used for fitting.

Standardisation: As usual, we standardise each feature to mean 0 and variance 1. This matters most for the two larger candidate lists below, whose features live on very different units (`loudness` in dB, `tempo` in BPM, the rest in $[0, 1]$): without it the Euclidean distance used by k -means would be dominated by loudness and tempo.

Feature selection: We compared three candidate feature lists over $k = 2, \dots, 10$ (Figure 9):

- **all audio** (7 features): `danceability`, `energy`, `loudness`, `acousticness`, `instrumentalness`, `valence`, `tempo`.
- **balanced** (5 features): the same list without `loudness` and `tempo`.
- **core** (3 features): `energy`, `acousticness`, `instrumentalness`.

We select between them using the silhouette, a metric that, bounded in $[-1, 1]$, can be compared directly across k -means fits of different dimension.

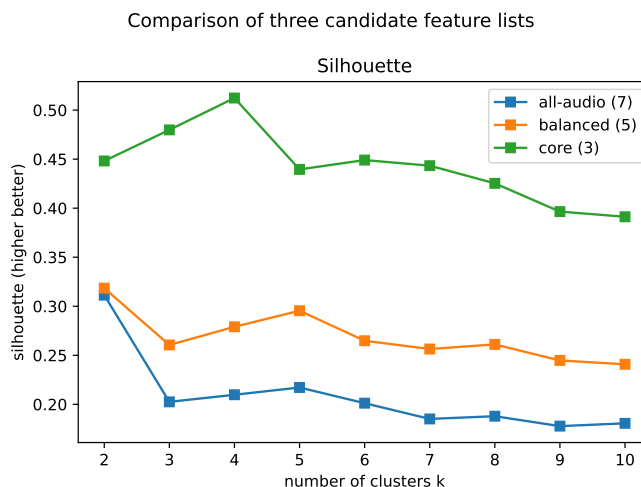


Figure 9: Comparison of the three candidate feature lists: `core` (3) has a clearly higher silhouette than the other two, across the whole range of k .

The result is clear: the more features we add, the more the clusters blur. We therefore cluster on the three core features (`energy`, `acousticness`, `instrumentalness`).

Choice of k : We run k -means for $k = 2, \dots, 10$ on the three core features. The metric used throughout is the (standardised) *Euclidean* distance. We use the three heuristics listed in TP 13 (section 2.2, “Algorithmic heuristics”) (Figure 10):

- **inertia (elbow):** sum of squared distances of points to their centroid. It always decreases with k , so we look for the “elbow” where it stops dropping fast.
- **silhouette** (higher is better): how close each point is to its own cluster versus the nearest other cluster, averaged over all points (in $[-1, 1]$).

- **Calinski–Harabasz** (higher is better): ratio of between-cluster to within-cluster dispersion.

Looking at Figure 10, $k = 4$ gives the best compromise between the three indices: the silhouette is at its maximum (0.52), the Calinski–Harabasz index, after its big jump from $k = 3$, is already close to its peak, and although the inertia curve has no sharply marked elbow, it has already dropped to 42% of its $k=2$ value and keeps decreasing more slowly from here. We therefore retain $k = 4$.

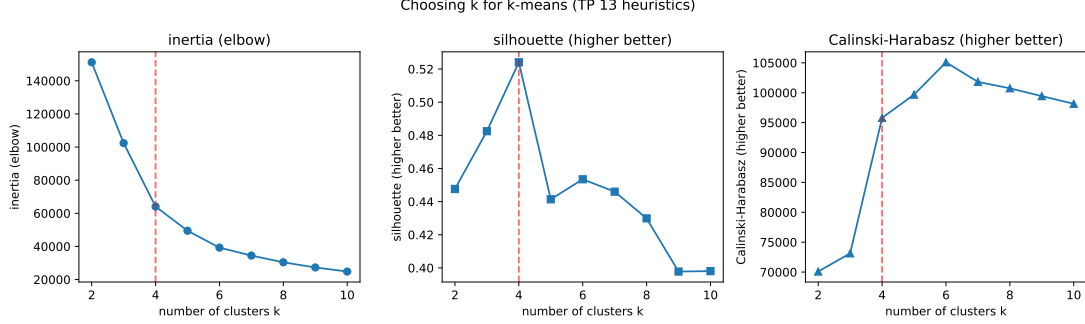


Figure 10: The three TP 13 heuristics vs k (red dashed line: $k = 4$).

Results and interpretation: The four clusters (silhouette 0.51) are musically coherent. Combining the heatmap (Figure 11), the values of the original centroids, and the most frequent genres in each cluster (not used during training), we can name each vibe:

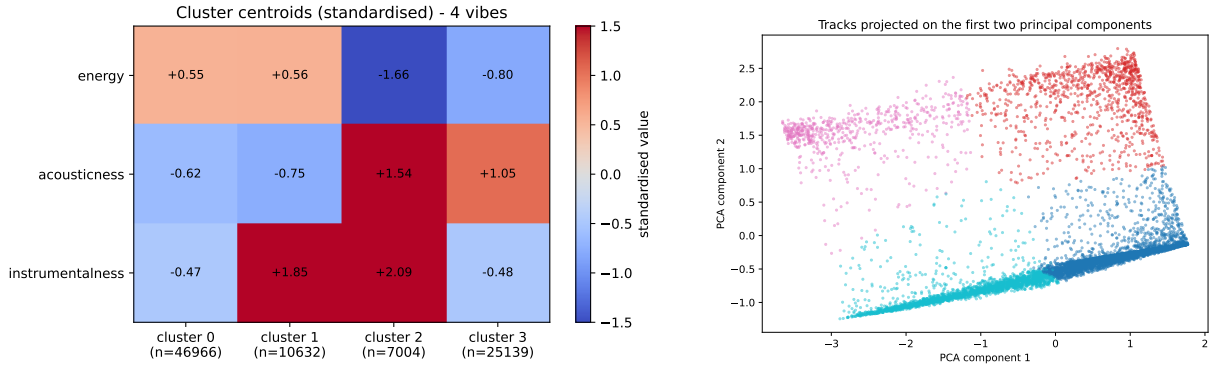


Figure 11: Left: heatmap of the standardised cluster centroids. Right: PCA projection on the first two principal components.

cluster (n)	vibe	energy	acoust.	instrum.	dominant genres
0 (47.0k)	Energetic vocal	0.78	0.12	0.02	heavy-metal, j-idol, dance, alt-rock, party
1 (10.6k)	Energetic instrumental	0.78	0.07	0.77	minimal/detroit-techno, grindcore, house
2 (7.0k)	Calm instrumental	0.21	0.85	0.85	new-age, ambient, study, classical, sleep
3 (25.1k)	Acoustic vocal	0.43	0.68	0.02	honky-tonk, comedy, tango, romance, can-topop

These four “vibes” are the kind of mood axes that streaming services use to seed automatic playlists, so the clustering is meaningful.

Evaluation: This is unsupervised learning: there is no ground truth to score against. `track_genre` cannot serve as one either, since 114 genres and our 4 clusters have no one-to-one correspondence. It is only useful qualitatively, to name the vibes above. We instead evaluate on two complementary criteria: how separated the clusters are, and how reproducible they are.

- **Internal validity:** the *silhouette* (0.51, near 1 is best) and the *Calinski-Harabasz* index both indicate that the clusters are compact and well separated from one another.
- **Stability (the main evaluation):** a clustering is only trustworthy if it is *reproducible*. Otherwise the four vibes could just be an accident of how *k*-means was initialised, or of which tracks happen to be in the dataset. We check this with the *Adjusted Rand Index* (ARI), which measures how much two clusterings correspond (1 = identical, 0 = unrelated). We test this in two ways: we re-run *k*-means from five different random seeds, which gives almost identical partitions (mean pairwise ARI = 0.999), and we re-cluster random 80% subsamples, which still agrees with the full clustering at ARI = 0.999. Both scores are close to 1, so the same four vibes come back regardless of the initialisation or the sample. They reflect a real structure in the data, not an artefact of the algorithm.